

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : Ashwin S

Reg no : 24011101007

Class : AI DS - A (batch 1)

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

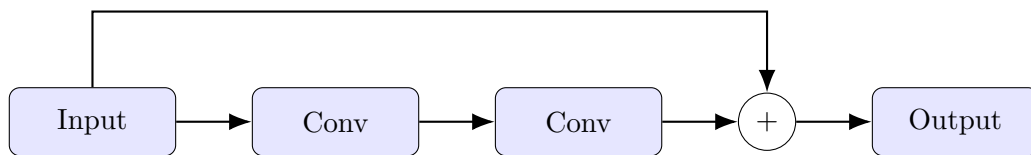


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

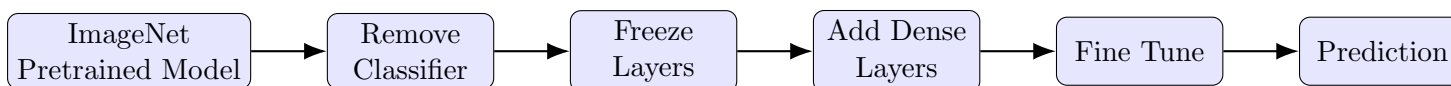


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$ (scaled to $160 \times 160 \times 3$)
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.
- Dataset Source : <https://www.kaggle.com/datasets/pankrzysiu/cifar10-python>

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

NOTE : The raw CIFAR-10 pixel values were retained (not pre-scaled to $[0,1]$) for the models feeding into a pretrained backbone, since each backbone's own `preprocess_input` function performs the ImageNet-specific normalization internally; pre-scaling before this step causes incorrect double normalization.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

NOTE : Input images were resized from $32 \times 32 \times 3$ to $160 \times 160 \times 3$ before being passed through the VGG16 convolutional base, since pretrained ImageNet backbones expect larger spatial input dimensions than the native CIFAR-10 resolution.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	20 (frozen base training)
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block (block5).
2. Train for another 8 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	3
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Four configurations were evaluated, each trained for 3 epochs on the VGG16 transfer-learning pipeline:

Config	LR	Batch Size	Optimizer	Dense Units	Frozen	Val. Accuracy
1	0.001	32	Adam	128	All	0.8533
2	0.0001	32	Adam	256	All	0.8478
3	0.001	64	SGD	128	All	0.8427
4	0.001	16	Adam	256	Partial	0.5451

Only 3 epochs were trained, considering computational time and resource constraints.

17. Source Code

The complete implementation, including data preprocessing, convolution and pooling experiments, model construction, training, transfer learning, evaluation, and the additional exercises, is available in the following repository :

<https://github.com/Ashwin271206/dl-lab/tree/main/Lab%204>

The repository contains a single Google Colab notebook covering all tasks experimented in this report, along with the generated figures and outputs.

18. Plots & Inferences

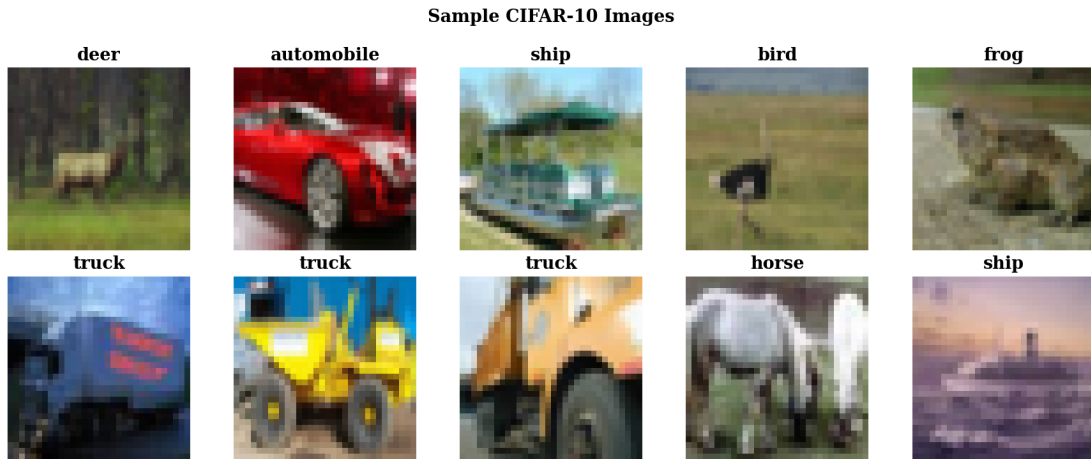


Figure 3: Sample CIFAR-10 Images

10 randomly sampled CIFAR-10 images are shown above, one label per image drawn from the ten classes. The samples confirm that the dataset was loaded and decoded correctly, with images at their native $32 \times 32 \times 3$ resolution.

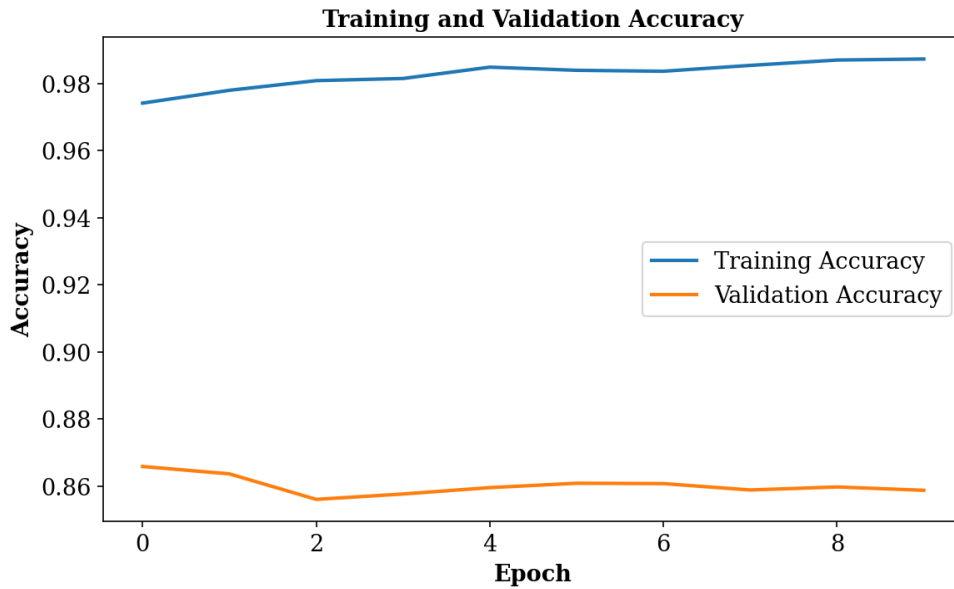


Figure 4: Training and Validation Accuracy (Frozen Base, VGG16)

Training accuracy rises steadily and stabilizes near 98% while the frozen convolutional base is used, showing the dense classifier head fitting the training data. Validation accuracy stays noticeably lower and flattens around 85–86%, indicating a persistent generalization gap during this phase.

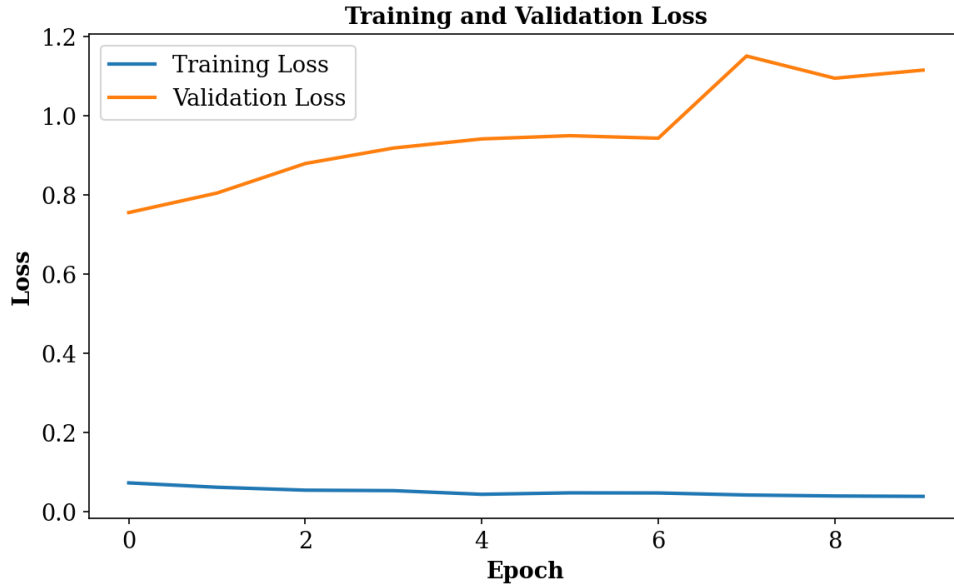


Figure 5: Training and Validation Loss (Frozen Base, VGG16)

Training loss decreases consistently across epochs, while validation loss rises after the first few epochs. This divergence between training and validation loss is a sign of overfitting when only the classifier head is being trained on frozen features.

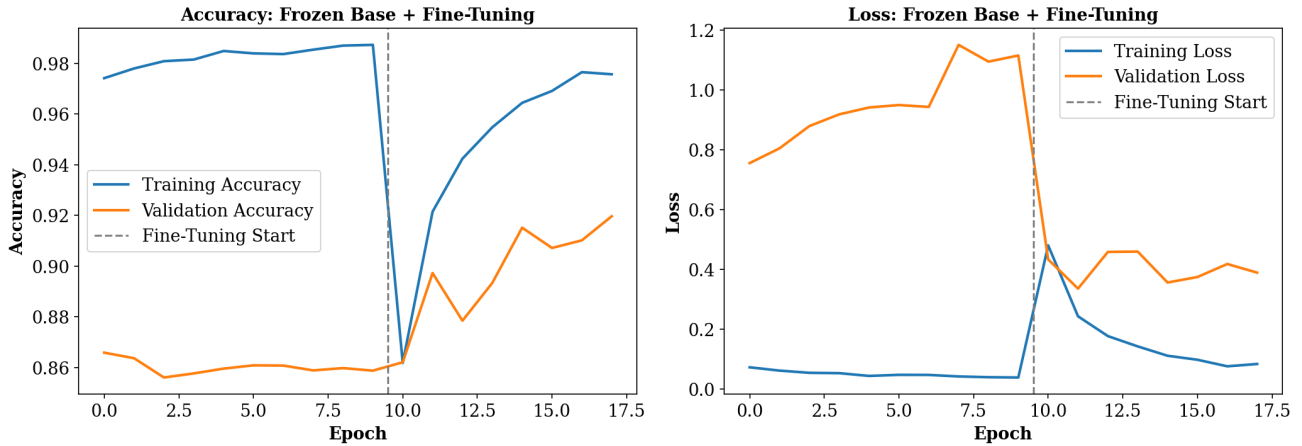


Figure 6: Combined Accuracy and Loss Across Frozen-Base and Fine-Tuning Phases

The dashed line marks the start of fine-tuning, after which validation accuracy improves markedly and validation loss drops sharply. This confirms that unfreezing the last convolutional block allows the network to adapt its ImageNet features to the CIFAR-10 domain, closing much of the earlier generalization gap.

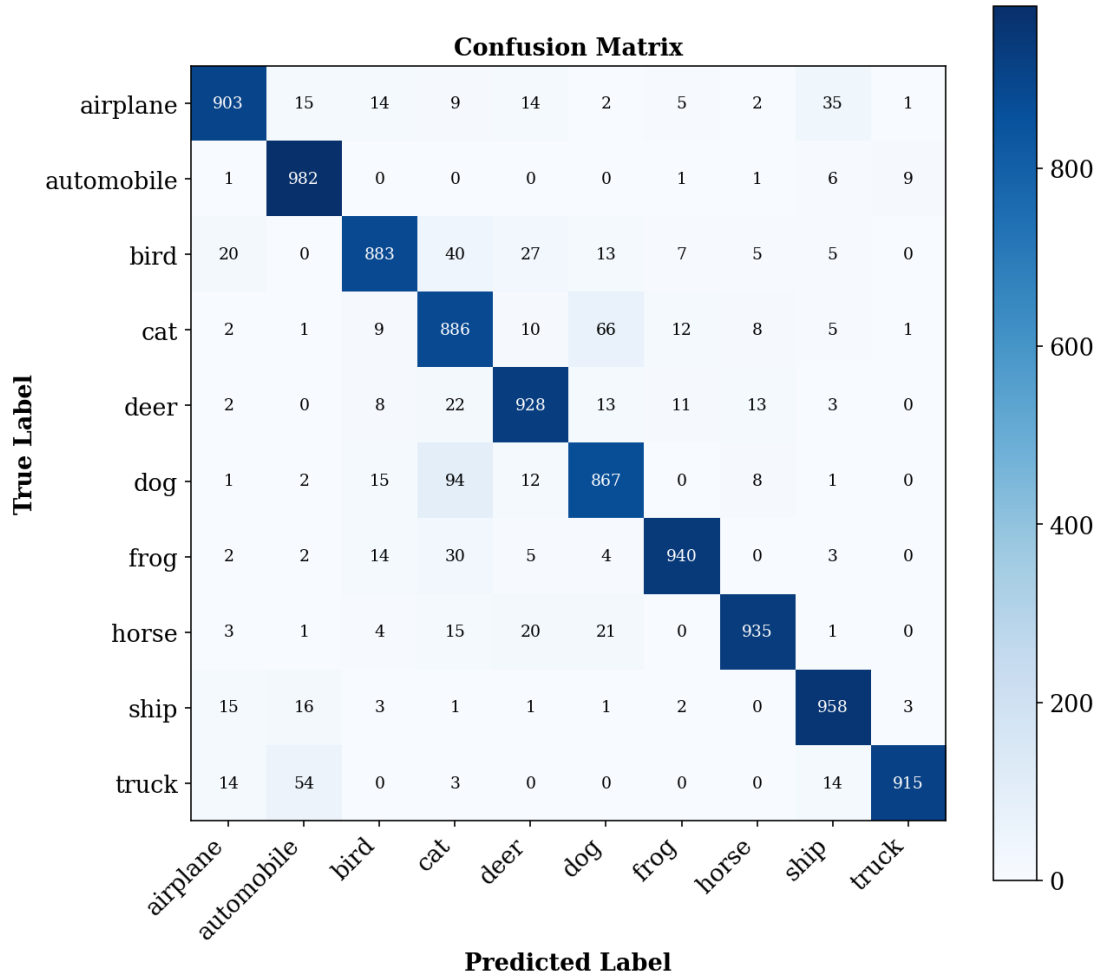


Figure 7: Confusion Matrix (Fine-Tuned VGG16 Model)

The confusion matrix shows strong diagonal dominance across all ten classes, with the highest confusion occurring between visually similar categories such as cat and dog. Vehicle classes such as automobile, ship, and truck are classified with very high accuracy.

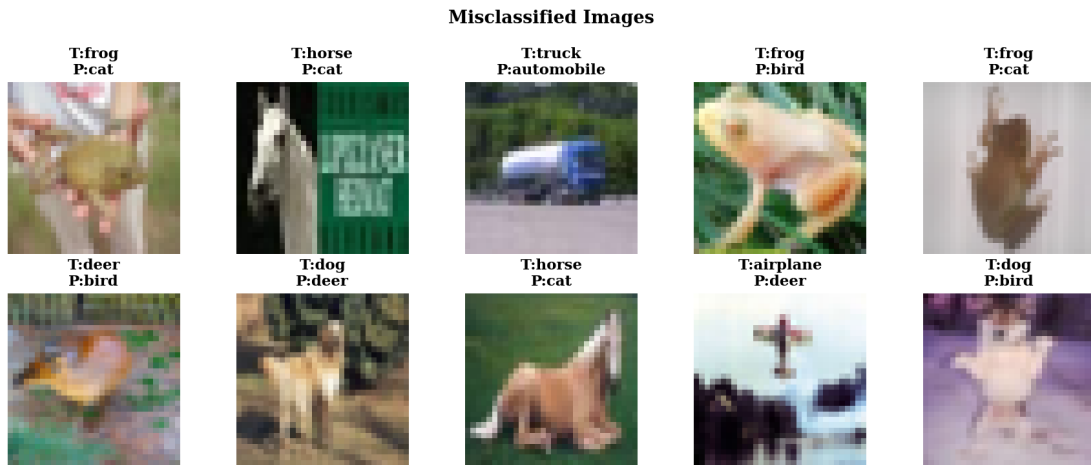


Figure 8: Misclassified Images (Fine-Tuned VGG16 Model)

The misclassified samples largely correspond to visually ambiguous cases, such as animals in unusual poses or partially occluded objects. This is consistent with the confusion matrix, where animal classes show the most inter-class confusion.

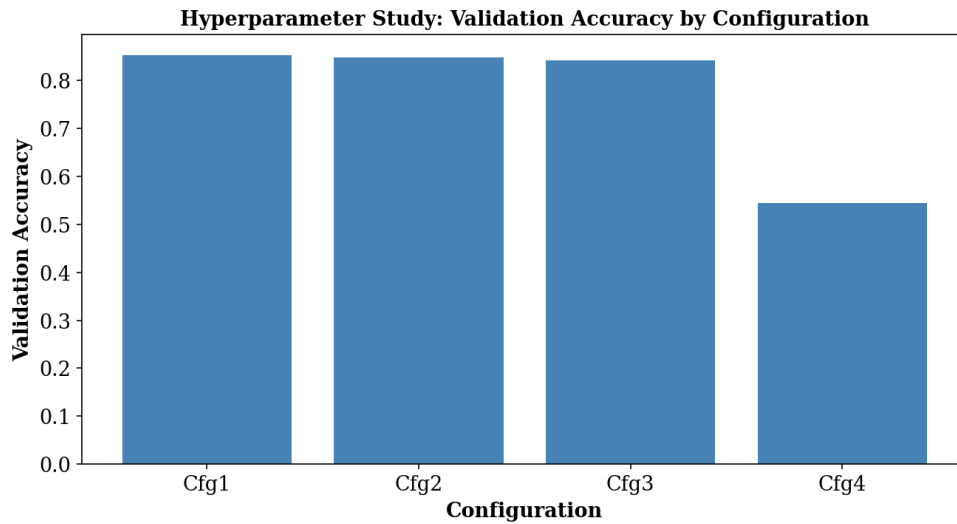


Figure 9: Hyperparameter Study: Validation Accuracy by Configuration

Configurations 1 through 3, all using a frozen convolutional base, achieve similar validation accuracy in the 0.84–0.85 range regardless of learning rate, batch size, or optimizer choice. Configuration 4, which partially unfreezes the base with only 3 epochs of training, performs substantially worse, showing that unfreezing layers needs more epochs to converge than a frozen-base setup.

19. Results

19.1 Performance Metrics

Metric	Value
Training Accuracy	97.572%
Testing Accuracy	91.970%
Precision	92.159%
Recall	91.970%
F1-score	92.005%
Training Time	4451.24 s
Total Parameters	14,848,586

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	53.78	70.48 s
AlexNet	8,576,906	70.61	290.36 s
VGG16	14,848,586	91.97	4451.24 s
GoogleNet (InceptionV3)	22,329,898	82.83	597.73 s
ResNet50	24,114,826	86.13	419.78 s

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was the first deep CNN to win the ImageNet Large Scale Visual Recognition Challenge by a large margin, demonstrating that deep convolutional networks could outperform traditional hand-engineered computer vision methods. It popularized ReLU activation for faster convergence, dropout for regularization, and GPU-based training for scaling to large networks and datasets, establishing the template that most modern CNNs still follow.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking multiple 3×3 filters achieves the same effective receptive field as a single larger filter (e.g., two 3×3 layers approximate a 5×5 receptive field) while using fewer parameters and adding more non-linear activation steps in between. This makes the network deeper and more expressive without a proportional increase in computational cost.

3. Explain the advantages of the Inception module.

The Inception module applies multiple filter sizes (1×1 , 3×3 , 5×5) and pooling in parallel within the same layer, allowing the network to capture features at multiple scales simultaneously. The 1×1 convolutions act as dimensionality reducers before the larger filters, cutting computational cost significantly while preserving representational power, resulting in a network with far fewer parameters than VGG16 for comparable depth.

4. What is the purpose of residual learning?

Residual learning reformulates each block to learn a residual function $F(x) = H(x) - x$ instead of directly learning the full mapping $H(x)$, with the output computed as $F(x) + x$. Skip connections let gradients flow directly through identity paths during backpropagation, which mitigates the vanishing gradient problem and makes it possible to train networks with far greater depth than would otherwise be feasible.

5. Differentiate LeNet and ResNet.

LeNet-5 is a shallow, 7-layer network with under 100K parameters designed for simple grayscale digit recognition using average pooling and tanh activations. ResNet50 is a 50-layer deep network with over 25M parameters that uses residual (skip) connections, batch normalization, and ReLU activations to enable stable training at much greater depth on complex, large-scale natural image datasets.

6. What is Transfer Learning?

Transfer learning is the technique of reusing a model that has already been trained on a large dataset (such as ImageNet) as the starting point for a new, related task, rather than training a new model from scratch. The pretrained convolutional layers, which have already learned general-purpose visual features like edges and textures, are retained and a new classifier head is trained on top for the target task.

7. Why is fine tuning required?

While a frozen pretrained base provides useful generic features, those features are optimized for the original dataset's domain (e.g., natural high-resolution ImageNet photos) and may not fully suit the new task's domain and image characteristics. Fine tuning unfreezes some of the later convolutional layers and continues training them at a low learning rate, allowing the network to adapt its higher-level features specifically to the target dataset and improve accuracy further.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated (atrous) convolution inserts gaps between kernel elements to increase the receptive field without adding parameters or reducing spatial resolution, and is commonly used in semantic segmentation. Transpose convolution, in contrast, performs learnable upsampling to increase the

spatial dimensions of a feature map, and is used in tasks like image generation, super-resolution, and autoencoder decoders.

9. Why do pretrained models converge faster?

Pretrained models start with weights that already encode useful low-level and mid-level visual features (edges, textures, shapes) learned from a large dataset, instead of starting from random initialization. This gives the optimizer a much better starting point, so fewer epochs are needed to reach good performance compared to training an equivalent network from scratch.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has roughly 60K parameters and 7 layers, making it extremely lightweight and fast to train even on limited hardware. ResNet50 has approximately 25.6M parameters across 50 layers with residual connections, making it far more computationally expensive to train and run inference on, though this added complexity is what enables it to model much more complex visual patterns.

21. Additional Exercises

21.1 Implement transfer learning using VGG16

This was implemented as the primary transfer-learning pipeline (Sections 15–19). The fine-tuned VGG16 model achieved a test accuracy of 91.97%, with performance metrics reported in Section 19.1.

21.2 Repeat the experiment using ResNet50

A separate transfer-learning pipeline was built using a frozen ResNet50 base with a Global Average Pooling and Dense classifier head, trained for 10 epochs.

Metric	Value
Test Accuracy	86.13%
Training Time	419.78 s
Total Parameters	24,114,826

ResNet50 converged considerably faster per epoch than VGG16 due to its smaller effective input resolution requirement and fewer trainable parameters in the frozen-base configuration, though its final accuracy remained below VGG16’s fine-tuned result.

21.3 Compare Adam and SGD optimizers

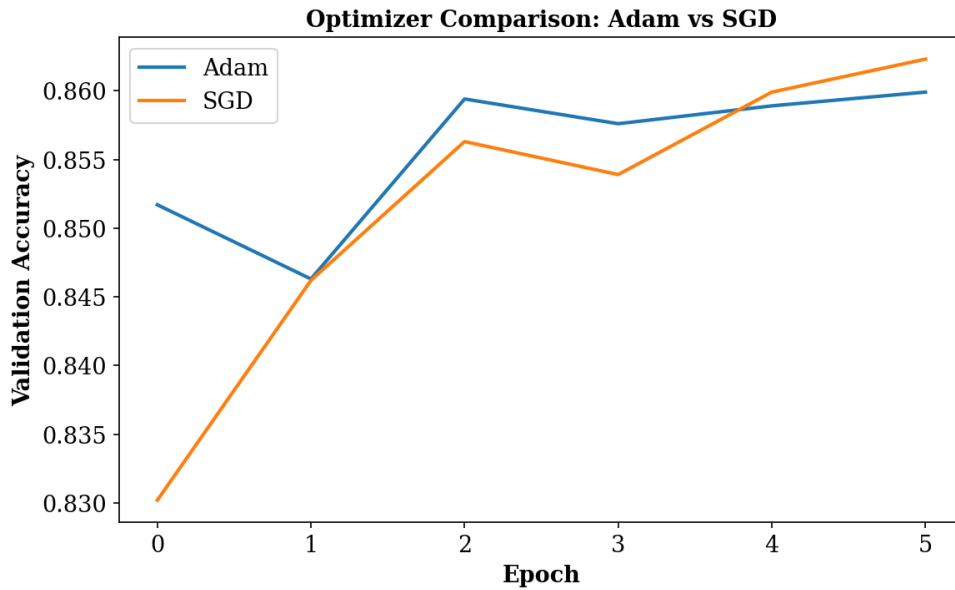


Figure 10: Validation Accuracy: Adam vs SGD

Adam reached a final validation accuracy of approximately 0.86 after 6 epochs, compared to approximately 0.86 for SGD as well, but Adam’s accuracy curve rose more quickly in the earlier epochs. This confirms Adam’s faster convergence behaviour due to its adaptive learning rate mechanism, even though both optimizers reach comparable final accuracy given enough epochs.

21.4 Compare training with frozen layers and fine tuning

Configuration	Final Validation Accuracy	Final Validation Loss
Frozen Convolutional Base	0.8588	1.1151
Fine-Tuned (block5 unfrozen)	0.9197	0.3894

Fine-tuning improved validation accuracy by roughly 6 percentage points and reduced validation loss by nearly two-thirds compared to the frozen-base configuration, confirming that allowing the last convolutional block to adapt its weights to CIFAR-10 substantially improves both accuracy and generalization.

21.5 Evaluate the model using Precision, Recall and F1-score

The fine-tuned VGG16 model was evaluated using macro-averaged Precision, Recall, and F1-score, computed with scikit-learn’s `precision_score`, `recall_score`, and `f1_score` functions.

Metric	Value
Precision (macro)	92.159%
Recall (macro)	91.970%
F1-score (macro)	92.005%

21.6 Compare the performance of LeNet, AlexNet and ResNet

LeNet-5 and AlexNet-style models were trained from scratch on CIFAR-10 (without pretrained weights), while ResNet50 used ImageNet-pretrained transfer learning.

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	53.78	70.48 s
AlexNet	8,576,906	70.61	290.36 s
ResNet50 (Transfer Learning)	24,114,826	86.13	419.78 s

ResNet50’s transfer-learning pipeline substantially outperformed both LeNet-5 and AlexNet trained from scratch, despite having far more parameters, demonstrating the advantage of pretrained ImageNet features over training a comparable-depth network from random initialization on a relatively small dataset like CIFAR-10. Between the two from-scratch models, AlexNet’s deeper architecture and larger capacity gave it a clear accuracy advantage over the shallower LeNet-5.

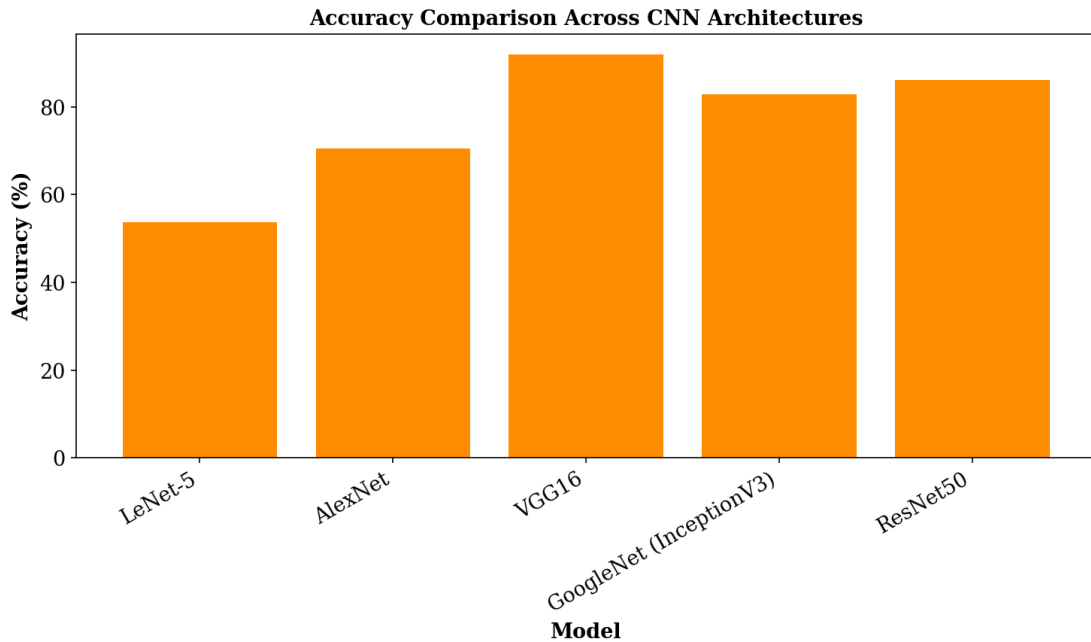


Figure 11: Accuracy Comparison Across CNN Architectures

VGG16 achieves the highest test accuracy among all five architectures after fine-tuning, followed by ResNet50 and GoogleNet (InceptionV3). LeNet-5 and AlexNet, trained from scratch without pretrained weights, achieve considerably lower accuracy, highlighting the benefit of transfer learning on a small, low-resolution dataset like CIFAR-10.

22. Conclusion

This experiment compared five deep convolutional neural network architectures – LeNet-5, AlexNet, VGG16, GoogleNet (InceptionV3), and ResNet50 – on the CIFAR-10 image classification task, using transfer learning for the pretrained architectures. The results confirm that pretrained models substantially outperform networks trained from scratch on a relatively small dataset: VGG16, fine-tuned on top of ImageNet weights, achieved the highest test accuracy of 91.97%, followed by ResNet50 at 86.13% and GoogleNet (InceptionV3) at 82.83%, while LeNet-5 and AlexNet trained from scratch reached only 53.78% and 70.61% respectively despite far fewer or comparable parameters. Fine-tuning the last convolutional block of VGG16 improved validation accuracy from 85.88% to 91.97%, demonstrating that

allowing pretrained features to adapt to the target domain yields a meaningful accuracy gain over a purely frozen feature extractor. The hyperparameter study further showed that learning rate, batch size, and optimizer choice had a comparatively minor effect on validation accuracy when the convolutional base remained frozen, whereas partially unfreezing layers without sufficient training epochs led to a clear drop in performance, underlining that the number of training epochs must scale with the number of trainable layers. Overall, the experiment reinforces those two central lessons: architectural depth alone does not guarantee better performance without sufficient training data or transfer learning, and leveraging pretrained ImageNet features – combined with careful fine-tuning – provides an efficient and effective route to strong classification accuracy on smaller, lower-resolution datasets such as CIFAR-10.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>